

An Exact String Matching Algorithm Based upon the Substring Encoding Method

H. M. Chen and R. C. T. Lee

Department of Computer Science and Information Engineering

National Chi Nan University

Puli, Nantou, Taiwan 545

{s94321543,rctlee}@ncnu.edu.tw

Abstract

The traditional exact string matching problem is to find all locations of a pattern string with length m in a text with length n . Here we propose a new encoding method to shorten the both lengths of pattern and text by substituting the substring between a special character for its length in $O(m+n)$. Then we use an exact matching algorithm to solve the exact string matching problem on the encoding pattern and text. As can be seen, by using the encoding method, the pattern and text can be shortened about $2/|\Sigma|$ times the lengths of the original ones. In practice, it performs better than $2/|\Sigma|$. For instance, for an English sentence pattern whose length is 20 and a text whose length is 5000, in average, the pattern is shortened to 16.5% of its original length and the text is shortened to 15.5% of its original length. Thus, the exact matching can be done in a much shorter time.

a case, we shall further do a further examination to make sure whether the answer is “yes” or not. If yes, of course, we still have to investigate whether P occurs in T .

Our idea also has the following characteristics:

1. The substring P' is coded in such a way that it becomes much shorter than P . The coded P' is denoted as P'_{en} .
2. The text T will also be coded by using the same coding method to code P' . The coded text is denoted as T'_{en} . It will be shown that T'_{en} is also shortened and will contain P'_{en} if and only if T contains P' .
3. Since both T'_{en} and P'_{en} are short now, it becomes easy to determine whether P'_{en} occurs in T'_{en} .

Section 1. The Introduction

In this paper, we are interested in the exact string matching problem which is defined as follows: We are given a text $T = t_1t_2 \cdots t_n$ and $P = p_1p_2 \cdots p_m$ and we are asked to find all occurrences of P in T . Much research has been done in this field [1, 2, 3, 4, 5, 6, 7, 8, R92, 9, 10, 11 and 12].

In this paper, we propose a new approach to solve this problem. Let us imagine that we select a substring P' in P . We first determine whether P' occurs in T . If the answer is “not”, we are certain that P does not occur in T . As we shall see later, our method will not give a “yes” answer. It will only give the answer that P' may occur in T . In such

Section 2. The Encoding Method

The first step of our method is to select a character x from P . Then we delete all characters to the first x in P and all characters to the right of the last x in P . The result is a substring enclosed by x . This substring must be of the form of $x^{a_1}S_1x^{a_2}S_2 \cdots S_kx^{a_{k+1}}$ for some k where x^i denotes a string of i x 's. This is the substring P' mentioned above.

For example, let $P = aacdstaaceedrcab$. Suppose we select character c . Then $P' = cdstaaceedrc$. We shall select x with the smallest frequency in P such that the size of the resulting P' is larger than 2. Consider $P = gbaacddccbbcc$. We will not use g , d and a . We will use b and obtain $P' = baacddccbb$.

Let d_i denote the size of S_i . Then

$P'_{en} = xd_1xd_2 \cdots xd_kx$. For example, for $P' = cdstaacceedrc$, $P'_{en} = c5cc4c$.

We now use the same method to encode T . Let $T = cactacgactactcgacgcta$ and $P = actacgact$. We select t which has the smallest frequency in P . Then $P'_{en} = t5t$. T is coded as T'_{en} by using the same coding method. Thus we obtain $T'_{en} = t5t2t6t$.

To give the reader some feeling about our algorithm, consider the following text T : “Under the wide and starry sky, dig the grave and let me lie. Home is the sailor, home from sea, and the hunter from the hill.” Suppose the pattern P is “Home is the sailor”. Let us assume that we select the character “i”. Then $P'_{en} = i6i$ and $T'_{en} = i16i18i6i6i36i$. We can now easily see that P'_{en} exists in T'_{en} .

Section 3. Matching and Examination

3.1 Matching Phase

After T and P have been encoded to T'_{en} and P'_{en} , we have to check whether P'_{en} occurs in T'_{en} or not. Thus this is an exact string matching problem. Any exact string matching algorithm [1, 2, 3, 4, 5, 6, 7, 8, R92, 9, 10, 11 and 12] can be used. But, since both text and pattern strings are much simplified, we recommend to use the quick search algorithm [S90]. We just show describe the outline here.

Suppose that P is aligned to a window of T as shown in Fig. 3-1 below. We perform a pair-wise comparing between text T and pattern P from left to right. Assume that the first mismatch occurs when comparing t_{s+i-1} with p_i . As shown, the character next to the last character of the window is x and assume that x appears in location j of P . We can shift the pattern $(m - j)$ positions to the right as shown in Fig.3-2. If x does not appear in P , we move $m + 1$ steps to the right.

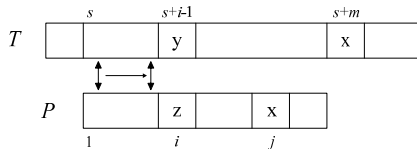


Fig. 3-1

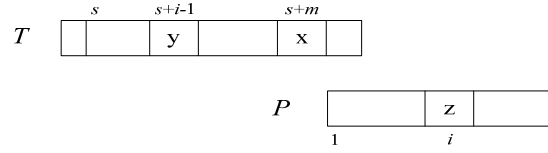


Fig. 3-2

Example 1: Let $T = ababcccabbabbcccccabb$ and $P = ccabbab$. We select the character “a” for our encoding. We select the substring $P' = abba$, and we have $P'_{en} = a2a$. Then, we encode T , $T'_{en} = a1a4a2a6a$.



$P'_{en} = a2a$
Mismatch occurs, slides P'_{en} 4 steps.

$T'_{en} = a1a4a2a6a$
 $P'_{en} = a2a$

Match occurs, record position 5 and slides P'_{en} 4 steps. Terminate the matching phase.

Example 2: Let $T = aaaaaatcacattaatcaaa$ and $P = aatcatctaadc$. Assume that we select character “c”, then we obtain $T'_{en} = c1c6c$ and $P'_{en} = c2c4c$. Now we can easily see that P'_{en} doesn't exist in T'_{en} . In such situation, we do not need to examine.

3.2 Examination Phase

If P'_{en} exactly matches starting at position i of T'_{en} , we need to know the corresponding position of T which should be examined. Let q denote most left side position of x in P , r denote most left side position of x in T . We define a function $Pos(i)$ to calculate the corresponding position of T while P'_{en} matches at position i of T'_{en} . We define $Pos(i)$ as following:

$$Pos(i) = \sum_{x \leq i} f(T'_{en}(x)),$$

$$\text{where } f(T'_{en}(x)) = \begin{cases} 1 & , \text{if } T'_{en}(x) \text{ is a character.} \\ T'_{en}(x) & , \text{if } T'_{en}(x) \text{ is a number.} \end{cases}$$

Thus, we align P to match with.

$$T_{r+Pos(i)-q, r+Pos(i)-q+m-1}$$

Example 3: Let $T = ababcccabbabbccccabb$ and $P = ccabbab$. Suppose from matching phase, we have $T'_{en} = a1a4a2a6a$, $P'_{en} = a2a$ and $m = 7$, we also record that P'_{en} occurs at position 5 of T'_{en} . The most left side position of a in P is 3, that is $q = 3$. The most left side position of a in T is 1, that is $r = 1$. We

calculate

$$Pos(5) = f(T'_{en}(1)) + f(T'_{en}(2)) + f(T'_{en}(3)) + f(T'_{en}(4)) + f(T'_{en}(5)) = 1 + 1 + 1 + 4 + 1 = 8$$

We need to examine whether P occurs at position $(1 + Pos(5) - q) = (1 + 8 - 3) = 6$ of T . That is, we need to align P to match with the substring $T_{6,12}$.

We align P to position 6 of T to verify whether there exists an exact string matching or not.

$T = ababcccabbabbccccabb$

$P = \quad \quad ccabbab$

Match occurs, we stop comparing P with T .

Example 4: Let $T = atctcagtcagaaccctggctc$ and $P = agaccgacc$. We can know that $T'_{en} = g3g6gg$ and $P'_{en} = g3g$ and $m = 9$. Suppose from matching phase, we know that P'_{en} occurs at position 1 of T'_{en} , $Pos(1) = 1$, $q = 2$, $r = 7$. We align P to match with the substring $T_{6,14}$.

$T = atctcagtcagaaccctggctc$



$P = \quad \quad agaccgacc$

Mismatch occurs, we stop comparing P with T .

Section 4. The Algorithm

Input: $T = t_1 t_2 \dots t_n, P = p_1 p_2 \dots p_m$

Output: All occurrences of P in T .

Step 1: Choose a character x whose frequency in P is as small as possible and does not obey the following cases.

Case 1: x only appears once in P .

Case 2: x appears twice and adjacent in P .

Step 2: Let P'_{en} be the longest substring of P that begins with x and ends with x . If the length of the substring between two nearest x is greater than 0, replace the substring with its length.

Step 3: Use the Quick Search Algorithm to find all occurrences of P'_{en} in T'_{en}

Section 5. The Property of the Encoded Method

The time complexity of encoding a string S is $O(|S|)$, where $|S|$ is the length of S . Assume that the character x is the best choice. We transform P and T into P'_{en} and T'_{en} respectively and this step can be done by linear scan.

In the following, we consider the worst case of our compression scheme. Let $|\Sigma|$ denote the alphabet size. Assume that the probability of each c in T and in P is equal for all $c \in \Sigma$.

Let $\alpha = \left\lfloor \frac{|T|}{|\Sigma|} \right\rfloor$ and $\beta = \left\lfloor \frac{|P|}{|\Sigma|} \right\rfloor$.

Suppose we select character x to

encode P'_{en} and T'_{en} , where the number of x in T is α and the number of x in P is β . The number of numerals in T'_{en} is at most $\alpha-1$, and that in P'_{en} is at most $\beta-1$. Thus

$$|T'_{en}| \leq \alpha + (\alpha - 1) = 2\alpha - 1 \quad \text{and}$$

$$|P'_{en}| \leq \beta + (\beta - 1) = 2\beta - 1. \quad \text{By definition,}$$

$$|T| \geq \alpha|\Sigma| \quad \text{and} \quad |P| \geq \beta|\Sigma|. \quad \text{Therefore, we}$$

$$\text{have} \quad \frac{|T'_{en}|}{|T|} \leq \frac{2\alpha - 1}{\alpha|\Sigma|} \cong \frac{2\alpha}{\alpha|\Sigma|} = \frac{2}{|\Sigma|} \quad \text{and}$$

$$\frac{|P'_{en}|}{|P|} \leq \frac{2\beta - 1}{\beta|\Sigma|} \cong \frac{2\beta}{\beta|\Sigma|} = \frac{2}{|\Sigma|}. \quad \text{This analysis}$$

shows that our compression scheme is quite effective. For the DNA type data, both text and pattern are compressed to half of their sizes. If the data are English language sentences, both text and pattern will be compressed to roughly 15% of their original sizes.

Section 6. Experiments

We wrote a program to implement our encoding method. In our experiments, we recorded the ratio of the size of the compressed data and that of the original data. Each result is the average of 100 experiments with different P 's and T 's.

Experiment 1: Natural Language Data

We randomly selected a part of an English article with length 5000 to be text T , and a part of it with length 10, 20, 30 and 40 to be pattern P . The results are shown in Table 6-1.

Table 6-1

$ P $	$ T $	$ P'_{en} $	$ T'_{en} $	$\Re$ of T	$\Re$ of P
10	5000	3.2	961.3	19.23%	32%
20	5000	3.3	775.2	15.5%	16.5%
30	5000	3.4	722.3	14.45%	11.33%
40	5000	3.6	737.3	14.75%	9%

$\Re$: compression ratio

Experiment 2: DNA Type Data

We randomly selected a part of a DNA sequence with length 5000 to be text T , and a part of it with length 10, 20, 30 and 40 to be pattern P . The results are shown in Table 6-2.

Table 6-2

$ P $	$ T $	$ P'_{en} $	$ T'_{en} $	$\Re$ of P	$\Re$ of T
10	5000	3.4	2377	34%	47.54%
20	5000	4.3	2048.9	21.5%	40.98%
30	5000	7.4	2246.8	24.67%	44.94%
40	5000	11	2310.4	27.5%	46.21%

$\Re$: compression ratio

Section 7. Conclusion Remarks

In this paper, we have proposed an encoding approach to compress pattern and text. The experiment shows that our algorithm is quite efficient. Further more, we plan to analyze the time complexity in average case. In the future, we also try to solve approximate string matching problem by using our approach.

Reference

- [1] Algorithms for finding patterns in strings, Aho, A. V., in *Handbook of Theoretical Computer Science*, Volume A, Algorithms and complexity, 1990, pp. 255-300.
- [2] Éléments d'algorithmique, Beauquier, D., Berstel, J. and Chrétienne, P., 1992, *Chapter 10*, pp. 337-377, Masson, Paris.
- [3] A new approach to text searching, Baeza-yates, R. A. and Gonnet, G. H., *Communication of the ACM*, Vol. 35, No. 10, 1992, pp. 74-82.
- [4] A fast string search algorithm, Boyer, R. S. and Moore, J. S., *Comm. ACM*, 20, 1977, pp. 762-772.
[BR92] Average running time of the Boyer-Moore-Horspool algorithm,

- Baeza-yates, R. A. and Régnier, M., *Theoretical Computer Science*, Vol. 92, No. 1, 1992, pp. 19-31
- [5] Very Fast String Matching Algorithm for Small Alphabets and Long Patterns, Christian, C., Thierry, L. and Joseph, D. P., *Lecture Notes in Computer Science*, Vol. 1448, 1998, pp. 55-64.
- [6] Practical fast searching in strings, HORSPOOL, R. N., *Software - Practice & Experience*, Vol. 10, 1980, pp. 501-506.
- [7] Fast string searching , HUME, A. and SUNDAY, D. M., *Software - Practice & Experience*, Vol. 21, 1991, pp. 1221-1248.
- [8] Tuning the Boyer-Moore-Horspool string searching algorithm, RAITA, T., *Software - Practice & Experience*, Vol. 22, No. 10, 1992, pp. 879-884.
- [9] A very fast substring search algorithm, Sunday, D. M., *Communications of the ACM*, Vol. 33, No. 8, 1990, pp. 132-142.
- [10] Experiments with a very fast substring search algorithm, SMITH, P. D., *Software - Practice & Experience*, Vol. 21, No. 10, 1991, pp. 1065-1074.
- [11] String matching algorithms and automata, SIMON, I., *1st American Workshop on String Processing*, 1993, pp. 151-157.
- [12] On improving the average case of the Boyer-Moore string matching algorithm, ZHU, R. F. and TAKAOKA, T., *Journal of Information Processing*, Vol. 10, No. 3, 1987, pp. 173-177 .